Experiment No.:- 10

Objective:- To implement N-Queens problem.

Theory

The **N-Queens Problem** is a classic problem in computer science.

• You are given:

o An **N × N chessboard**

o **N queens**

• Goal:

Place all N queens on the board such that:

▪ No two queens attack each other

Conditions:

• No two queens share the same row

• No two queens share the same column

• No two queens share the same diagonal

Approach: Backtracking

We place queens row by row and check if placing a queen is safe.

Safety Check Conditions:

• No queen in same column

• No queen in left diagonal

• No queen in right diagonal

If safe → place queen

Else → try next column

Algorithm

1. Start
2. Input value of N
3. Initialize an empty chessboard
4. Place queens row by row using recursion
5. For each column:
   - o Check if position is safe
   - o If safe → place queen
   - o Recur for next row
   - o If fails → backtrack (remove queen)
6. If all queens placed → print solution
7. Stop.

Program:

```
#include <stdio.h>
#define MAX 20
```

```
int board[MAX];

// Function to check if queen can be placed
int isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        // Same column or diagonal
        if (board[i] == col ||
            board[i] - i == col - row ||
            board[i] + i == col + row) {
            return 0;
        }
    }
    return 1;
}

// Function to solve N-Queens using backtracking
void solveNQueens(int row, int n) {
    if (row == n) {
        printf("\nSolution:\n");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i] == j)
                    printf(" Q ");
                else
                    printf(" . ");
            }
            printf("\n");
        }
        return;
    }

    for (int col = 0; col < n; col++) {
        if (isSafe(row, col)) {
            board[row] = col; // place queen
            solveNQueens(row + 1, n);
        }
    }
}

int main() {
    int n;

    printf("Enter number of queens: ");
```

```
    scanf("%d", &n);

    solveNQueens(0, n);

    return 0;
}
```

Example Input
Enter number of queens: 4
Example Output
Solution:
. Q . .
. . . Q
Q . . .
. . Q .

Time Complexity
• Worst case: **O(N!)**
(Backtracking explores all possible configurations)

Result
The N-Queens problem was successfully implemented using **Backtracking**. The algorithm efficiently finds all possible ways to place N queens on the chessboard such that no two queens attack each other.